PNP Formal Reconstruction Report

Compiled Lean theorem inventory and current claim boundary

Coordinate `PNP-FORMAL-RECONSTRUCTION-STATUS-2026-07-13-28`

The repository does not currently establish $P = NP$.

Public theorem emission is disabled. A direct finite raw-machine verifier now proves concrete CNF-SAT membership in NP with an explicit polynomial bound. It does not prove CNF-SAT in P, NP-hardness, NP-completeness, or $P = NP$. The general charged pipeline has not been linked to the raw machine kernel, its reviewed activation fingerprints are unset, and no root theorem exists. The abstract string-handle bridge is never an activation source.

Compiled declarations	5197
Theorem-kind declarations	2197
Assumption-free theorem-kind declarations	2096
Project-specific axioms	4
Concrete publication gate passed	**false**

Inventory SHA-256: `fcc6b1b8133562a155455f29d9834910b58ef3584a49fb5ca2d428a2e9515121`

Generated from the pinned compiled Lean environment, not from source-text parsing.

Contents

1 Current authority and claim boundary

This report is the current repository report surface. Its factual theorem inventory comes from the compiled `PNP` environment under `leanprover/lean4:v4.31.0`. The exporter enumerates `Environment.constants`, resolves each originating module, and calls `Lean.collectAxioms` for every exported project declaration. The committed inventory is `status/LEAN_THEOREM_INVENTORY.json`; the public mirror is byte-identical.

The target remains $P = NP$, but target wording—including the new inactive definition `PNP.Main.ConcretePEqualsNP`—is not theorem evidence. JavaScript acceptance, Boolean fields, JSON strings, historical release records, report prose, and external review cannot activate a theorem. Only the separately derived concrete publication gate may control theorem emission, and that gate is currently false.

The completed direct CNF verifier consumes the literal canonical pair of an encoded formula and assignment certificate. Lean proves universal acceptance equivalence, rejection equivalence, and absence of timeout at its explicit polynomial fuel bound, and constructs a proof-bearing `PolynomialTimeVerifier CNFSAT`. Thus `PNP.Concrete.CNFSAT` is in the concrete NP class. This is a verifier theorem, not a deterministic polynomial-time SAT algorithm.

Machine field	Current value
`mathematicalTheoremEstablished`	**false**
`publicTheoremEmissionAllowed`	**false**
`finalTheoremReady`	**false**
`publicTheoremStatement`	`null`
`rootLeanTheorem`	`PNP.Main.p_eq_np`
`rootLeanTheoremPresent`	**false**

2 Concrete publication gate

The compatibility entry is `PNP.Main.p_eq_np`; a matching name alone is insufficient. The separate publication target is `PNP.Main.ConcretePEqualsNP`. Eligibility also requires the exact theorem type, reviewed kernel fingerprints, and a tightly fixed Lean-standard axiom closure. The target is present as an inactive definition backed by finite charged-pipeline semantics; its presence is not a proof, and the compiler/refinement link to the raw machine kernel remains an explicit blocker for the general charged-pipeline target. The direct CNF verifier is one closed raw-machine instance; it does not supply that general link.

The expected fingerprints are intentionally unset. This is a fail-closed migration gate, not a missing runtime input. A verifier must reject any attempt to set `passed=true` while a fingerprint is null, while the concrete model is ineligible, or while any other subcheck is false.

Gate subcheck	Result
standardComplexityModelEligible	**false**
concreteTargetPresent	**true**
concreteTargetIsDefinition	**true**
concreteTargetKernelTypeFingerprintConfigured	**false**
concreteTargetKernelTypeFingerprintMatches	**false**
concreteTargetKernelValueFingerprintConfigured	**false**
concreteTargetKernelValueFingerprintMatches	**false**
compatibilityRootPresent	**false**
compatibilityRootIsTheorem	**false**

Gate subcheck	Result
compatibilityRootHasExactConcreteType	**false**
compatibilityRootKernelTypeFingerprintConfigured	**false**
compatibilityRootKernelTypeFingerprintMatches	**false**
axiomClosureFingerprintConfigured	**false**
axiomClosureFingerprintMatches	**false**
sourceClosureFingerprintConfigured	**false**
sourceClosureFingerprintMatches	**false**
axiomClosureUsesOnlyLeanStandardAllowlist	**false**

Allowed foundation axioms are fixed to `Classical.choice`, `Quot.sound`, and `propext`. Project axioms, `sorryAx`, or any unknown axiom make the gate fail. The four currently disclosed project axioms are:

- `PNP.CheckPCCPackexp`
- `PNP.GeneratePCCPack`
- `PNP.LockedNANDThreshold`
- `PNP.ResidualBandExactMinimization`

2.1 Why the abstract bridge is ineligible

The current `PNP.PEqualsNP` abbreviation compares witness classes whose language and algorithm structures carry string names or code handles rather than concrete execution semantics and proved polynomial bounds. Consequently, even a kernel-clean theorem of that abstract type would not establish the standard P-versus-NP statement. It is compatibility information only.

The separate `PNP.Concrete.PEqualsNP` target uses proof-bearing P and NP witnesses, canonical input/certificate pairing, finite program syntax, and charged polynomial runtime and output bounds. This is a substantive formalization milestone, but it is not yet proved equivalent to execution in the raw machine kernel. SAT completeness, SAT in P, and the root theorem also remain absent.

3 Formal milestone ledger

Each earned row requires exact compiled names and theorem kinds, empty axiom closures, per-name domain-separated kernel-type SHA-256 matches for 157 detailed candidates, and the pinned whole-Lean source closure. Human-readable scope remains conservative: type or source drift revokes credit.

Milestone	Status	Exact scope	Boundary
Milestone	Status	Exact scope	Boundary
Concrete machine and cost kernel	formalized-foundation-only	One literal finite input framer handles every raw bitstring locally. It uses exactly 4 work steps on empty input, 4 * k * k + 9 * k + 7 for k complete two-bit cells, and 4 * k * k + 9 * k + 5 when the final cell is partial; from ordinary raw startConfig it accepts without timeout within 6 * m * m + 39 * m + 75 and is blank-equivalent to the represented endpoint. All 70 input-framer declarations have empty axiom closure. Every supplied exact n-step raw target run lifts to exactly 3 * n successful work steps. The bridge module adds symbol-preserving framer-to-simulator and verdict-indexed handoff launches for canonical paired inputs; the terminal bridge appends two disjoint packer copies to one finite work table. PipelinePairedCompiler compiles that complete canonical-pair table to one raw machine. For every proof-bearing PolynomialTimeMachine and every canonical pair, it internally extracts an exact target prefix k <= p(m), proves target output length at most B(m) = m + p(m) + 1, and pads the complete execution to R(m) = inputFramerRawTimeBound(m) + 6 + 18*p(m) + 6 + framedOutputHandoffRawTimeBound(B(m)) + terminalBridgeRawTimeBound(B(m)). The compiled bounded verdict, no-timeout result, language acceptance, and ordinary machineOutput are exact. All 28 paired-compiler declarations, 59 terminal-bridge declarations, and 69 terminal-packer declarations have empty axiom closure.	The all-input theorem stops at the accepting framer frame. The only complete four-stage compiler remains restricted to canonical BitString.pair inputs: no theorem transports an arbitrary empty, odd, trailing, malformed, or other non-pair input through the renamed simulator, handoff, and terminal packer. Uniform all-input FunctionProgram.RawRefinement and DecisionProgram.RawRefinement therefore remain absent. The concrete complexity machine-link blocker remains open; CNF-SAT in P, NP-completeness, and P = NP are not established. PNP.Main.p_eq_np remains absent and the publication gate remains false.

Milestone	Status	Exact scope	Boundary
Concrete P, NP, and polynomial reductions	formalized-with-machine-link-pending	Bitstring languages, finite charged decision/verifier/function pipelines grounded in concrete machines, polynomial certificate/runtime/output bounds, exact raw-machine refinement contracts and machine-leaf witnesses, many-one reductions, and the NP-complete-in-P implication.	The refinement contracts and leaf cases do not compile function composition or decision precomposition, construct raw paired verifiers, prove charged/raw class equivalence, discharge the general machine-link blocker, prove CNF-SAT is in P or NP-complete, or establish the publication root theorem.
Concrete universal CNF-SAT verifier	formalized-np-membership-only	An unconditional formula-grammar outcome, universal bounded work outcome, exact accept/reject semantics, no-timeout theorem, literal finite-machine paired verifier, and CNF-SAT membership in NP.	This proves CNF-SAT membership in NP only; it does not prove CNF-SAT is in P, NP-completeness, or P = NP.
Typed direct-wire NAND semantics	formalized	Typed topological Boolean NAND programs and ordered multi-output direct-wire semantics.	This does not establish circuit minimization, SAT, or P = NP.
Finite enumeration, equivalence, and reference minimum	formalized	Exhaustive finite Boolean direct-wire search under the empty-profile reference model.	No polynomial-runtime result is obtained from this exhaustive reference search.
Concrete framed replacement and slack	formalized	The concrete serial framed context with support outputs and explicit bypass wires.	This is not an arbitrary-support replacement theorem for the report's global family.
Locked-NAND local candidates and baseline accounting	formalized	Typed local macro candidates, source-derived counts, and five finite local square baselines.	Local minima are not a global BaselineDistinct or locked-NAND threshold theorem.
Conditional locked-NAND threshold boundary	formalized-with-premises	A proof-bearing six-premise candidate boundary for an arbitrary satisfiable proposition.	The premises are not instantiated by a uniform SAT-to-locked-NAND builder.
Fail-closed explicit-list residual routes	formalized-explicit-list-only	Executable strict-gain search over a caller-supplied finite implementation list.	Unresolved excludes no gain outside the supplied list and cannot imply ZeroSlack.
Global locked-NAND construction and threshold	not-formalized	A uniform answer-independent SAT instance builder, global baseline, trace equivalence, and threshold.	The conditional boundary does not discharge this missing theorem.
Global ZeroSlack, PCCMin, and polynomial runtime	not-formalized	Complete residual routing, global ZeroSlack contradiction, exact minimization, and polynomial bounds.	Proof-bearing result structures and explicit-list failure do not supply global completeness.

Milestone	Status	Exact scope	Boundary
Concrete standard P-versus-NP target and root theorem	not-formalized	Raw-machine-linked complexity classes, concrete SAT completeness and a SAT decider, plus the publication root theorem.	The concrete target definition remains inactive: CNF-SAT now has a raw-machine verifier and NP-membership proof, but a deterministic polynomial-time CNF-SAT decider, NP-completeness transport, and PNP.Main.p_eq_np remain absent; the legacy string-handle bridge is ineligible.

4 Compiled inventory summary

The inventory contains 5197 exported `PNP.*` kernel declarations from 48 modules. It excludes 1032 private compiler auxiliaries. Counts describe the compiled environment; they do not measure mathematical completeness.

Module	Declarations	Theorems
PNP.Bridge	93	22
PNP.Complexity	93	20
PNP.Concrete.BitString	123	60
PNP.Concrete.CNF	247	60
PNP.Concrete.CNFVerifier	10	7
PNP.Concrete.CNFWorkCorrectness	859	525
PNP.Concrete.CNFWorkFrameCorrectness	16	4
PNP.Concrete.CNFWorkInput	29	14
PNP.Concrete.CNFWorkMachine	85	3
PNP.Concrete.CNFWorkTransitions	64	63
PNP.Concrete.CNFWorkUniversalCorrectness	294	145
PNP.Concrete.Complexity	312	92
PNP.Concrete.Machine	284	67
PNP.Concrete.PipelineInputFramer	106	18
PNP.Concrete.PipelineMachineSimulation	175	79
PNP.Concrete.PipelineOutputHandoff	23	4
PNP.Concrete.PipelinePairedCompiler	29	25
PNP.Concrete.PipelineRefinement	46	15
PNP.Concrete.PipelineStageBridges	77	59
PNP.Concrete.PipelineStateNamespace	77	34
PNP.Concrete.PipelineTapeGeometry	20	12
PNP.Concrete.PipelineTerminalBridge	73	62
PNP.Concrete.TapeBlankEquivalence	25	17
PNP.Concrete.TapeHandoff	18	11
PNP.Concrete.Target	2	1
PNP.Concrete.TerminalOutputPacker	111	36
PNP.Concrete.WorkInput	23	14
PNP.Concrete.WorkMachine	308	108
PNP.DirectWireBaseline	48	25
PNP.LockedNAND	11	4
PNP.LockedNANDBaseline	57	22
PNP.LockedNANDDirect	84	57
PNP.LockedNANDLocalBaseline	30	22
PNP.LockedNANDMacros	288	98
PNP.LockedNANDPrefix	84	40
PNP.LockedNANDThresholdBoundary	60	35
PNP.Main	36	12
PNP.NANDComposition	77	38
PNP.NANDEnumerator	120	46
PNP.NANDMinimum	56	23

Module	Declarations	Theorems
PNP.NANDSemantics	198	80
PNP.NANDSlack	15	12
PNP.NANDTruthTable	72	28
PNP.PCCMin	44	8
PNP.ResidualBand	9	2
PNP.ResidualRoutes	141	46
PNP.SAT	4	1
PNP.ZeroSlack	141	21

5 Remaining formal blockers

Seven formal blockers remain active:

1. `Formal.ConcreteComplexityMachineLink`
2. `Formal.ConcreteSAT`
3. `Formal.LockedNANDThreshold`
4. `Formal.ResidualBandMinimizer`
5. `Formal.ZeroSlack`
6. `Formal.PolynomialRuntimeAndCertificateBounds`
7. `Formal.RootTheoremAndAxiomAudit`

The conditional locked-NAND threshold module assumes typed baseline and full candidates, baseline conditions, preservation of existing outputs, an unsatisfiable final-zero law, and satisfiable final-output laws. The explicit residual scanner searches only a caller-supplied finite list. Neither result constructs the missing global reduction or proves global ZeroSlack, polynomial PCCMin, SAT in P, or P equals NP.

6 Historical report provenance

Earlier 56-page report revisions stated a direct checker-mediated P-versus-NP claim. Those bytes remain historical audit material at their pinned legacy coordinates and through the explicit archive and supersession records. They are not imported into, quoted as authority by, or used to generate this report. File identity and replay of historical predicates do not establish the named mathematical propositions.

Source tag	`final-pnp-proof-report-hardened-7072f8d`
Source commit	`7072f8d0bda6d44d240f9bb3fad624fd357e1278`
Archive manifest	`archive/legacy-v0/ARCHIVE.json`

7 Verification

The current reconstruction can be checked with:

```
lake build PNP
node scripts/export-lean-theorem-inventory.mjs --check
node scripts/generate-formal-publication.mjs --check
node pcc-formal-reconstruction-status0.mjs --json
npm run pnp:verify -- --no-write
npm run report:check
```

The inventory coordinate is `PNP-LEAN-THEOREM-INVENTORY-2026-07-13-28`. Its exact byte digest is `fcc6b1b8133562a155455f29d9834910b58ef3584a49fb5ca2d428a2e9515121`. The report is regenerated from that inventory and the reviewed publication map. The reviewed milestone source-closure SHA-256 is `6b6558134e01809a69181416bf121cf3250dee4e468e6bb0efd7351cb5e5b1a6`; the computed and pinned values match. Successful regeneration confirms consistency of these status surfaces; it does not turn an absent concrete theorem into a proof.